# Practice Interview

## Objective

**The partner assignment aims to provide participants with the opportunity to practice coding in an interview context. You will analyze your partner's Assignment 1. Moreover, code reviews are common practice in a software development team. This assignment should give you a taste of the code review process.**

## Group Size

Each group should have 2 people. You will be assigned a partner

## Parts:

- Part 1: Complete 1 of 3 questions
- Part 2: Review your partner's Assignment 1 submission
- Part 3: Perform code review of your partner's assignment 1 by answering the questions below
- Part 3: Reflect on Assignment 1 and Assignment 2

## Part 1:

**You will be assigned one of three problems based of your first name. Enter your first name, in all lower case, execute the code below, and that will tell you your assigned problem. Include the output as part of your submission (do not clear the output). The problems are based-off problems from Leetcode.**

In [1]:
```
import hashlib

def hash_to_range(input_string: str) -> int:
     hash_object = hashlib.sha256(input_string.encode())
     hash_int = int(hash_object.hexdigest(), 16)
     return (hash_int % 3) + 1
input_string = "feihong"
result = hash_to_range(input_string)
print(result)
```

1

▶ Question 1

### Starter Code for Question 1

In [ ]:
```
# Definition for a binary tree node.
# class TreeNode(object):
#     def __init__(self, val = 0, left = None, right = None):
#         self.val = val
#         self.left = left
#         self.right = right
```

```
def is_duplicate(root: TreeNode) -> int:
  # TODO
```

In [2]:
```
from collections import defaultdict

class TreeNode(object):
    def __init__(self, val = 0, left = None, right = None):
        self.val = val
        self.left = left
        self.right = right

def is_duplicate(root: TreeNode) -> int:
    def serialize(node):
        if not node:
            return "#"
        serial = f"{node.val},{serialize(node.left)},{serialize(node.right)}"
        count[serial] += 1
        return serial

    count = defaultdict(int)
    serialize(root)

    for freq in count.values():
        if freq > 1:
            return 1  # Found a duplicate subtree
    return 0  # No duplicates
```

▶ Question 2

### Starter Code for Question 2

In [ ]:
```
# Definition for a binary tree node.
# class TreeNode(object):
#     def __init__(self, val = 0, left = None, right = None):
#         self.val = val
#         self.left = left
#         self.right = right
def bt_path(root: TreeNode) -> List[List[int]]:
  # TODO
```

▶ Question 3

### Starter Code for Question 3

In [ ]:
```
def missing_num(nums: List) -> int:
  # TODO
```

## Part 2:

You and your partner must share each other's Assignment 1 submission.

## Part 3:

Create a Jupyter Notebook, create 6 of the following headings, and complete the following for your partner's assignment 1:

- Paraphrase the problem in your own words.

# Your answer here

My partner is assigned to Question Three: Move All Zeros to the End (Simplified Notes) What to do: You are given a list of numbers. Some of the numbers are 0. Your task is to move all the 0s to the end of the list. But important: keep the same order for all the other (non-zero) numbers.

- Create 1 new example that demonstrates you understand the problem. Trace/walkthrough 1 example that your partner made and explain it.

# Your answer here

New example: Input: nums = [0, 2, 0, 0, 1, 5] Output: [2, 1, 5, 0, 0, 0]

Move all non-zero numbers to the front: 2, 1, 5

Then fill the rest with zeros

Step-by-step:

insert_pos = 0

Loop through each number:

0 → skip (because it's zero)

1 → put at nums[0], now nums = [1, 1, 0, 3, 12], insert_pos = 1

0 → skip

3 → put at nums[1], now nums = [1, 3, 0, 3, 12], insert_pos = 2

12 → put at nums[2], now nums = [1, 3, 12, 3, 12], insert_pos = 3

Now fill the rest with zeros starting from position 3:

nums[3] = 0, nums = [1, 3, 12, 0, 12]

nums[4] = 0, nums = [1, 3, 12, 0, 0]

- Copy the solution your partner wrote.

In [ ]:
```python
# Your answer here
from typing import List

def move_zeros_to_end(nums: List[int]) -> List[int]:
    # TODO
    length_nums = len(nums)
    i = 0
```

```
    while i < length_nums:
        if nums[i] == 0:
            nums.pop(i)
            nums.append(0)
            length_nums -= 1
        else:
            i += 1
    return nums
```

- Explain why their solution works in your own words.

# Your answer here

My partner's code works because it does the task in two parts:

Move non-zero numbers to the front The code looks at each number in the list one by one. If the number is not zero, it puts it at the front (in the correct position) and moves to the next spot. This keeps the order of non-zero numbers the same.

Put zeros at the end After moving all the non-zero numbers, the rest of the list is filled with zeros. This makes sure that all zeros are at the end.

Because of these two steps, the final result is correct — all non-zero numbers stay in order, and all zeros are at the end.

- Explain the problem's time and space complexity in your own words.

# Your answer here

Time Complexity The code goes through the list two times:

First, it looks at every number to move non-zero numbers to the front.

Then, it fills the rest of the list with zeros.

Each pass checks every number once. So if the list has n numbers, the total steps are about 2n, which is still considered O(n).

Time complexity is O(n) — where n is the number of elements in the list.

Space Complexity The code does not use any extra list or array. It changes the original list directly (in-place), and only uses one extra variable (insert_pos).

Space complexity is O(1) — constant space, because no extra memory is used for another list.

✅ Summary (Simple Words) Time: O(n) → because we check each number two times. Space: O(1) → because we don't create a new list, just use one variable.

- Critique your partner's solution, including explanation, and if there is anything that should be adjusted.

# Your answer here

My Partner’s Code – Good and Bad Points ✅ What is Good The code is correct. It moves all the zeros to the end and keeps the order of other numbers.

It is fast. The code checks each number only two times. That means the speed is good (O(n)).

It uses little memory. It does not make a new list. It changes the original list, so memory use is low (O(1)).

⚠️ What Can Be Better Better variable name. The name insert_pos is okay, but maybe position or index is easier to understand.

No comments. The code has no comments. Adding small notes in the code will help others understand it better.

Clear return or not. The function changes the list in-place. If it returns the list, it should be clear. → Suggestion: Add a short comment saying "this changes the list directly".

## Part 4:

Please write a 200 word reflection documenting your process from assignment 1, and your presentation and review experience with your partner at the bottom of the Jupyter Notebook under a new heading "Reflection." Again, export this Notebook as pdf.

### Reflection

# Your answer here

## Reflection

In Assignment 1, I started by reading the problem carefully to make sure I understood the goal. I tested some examples before writing any code. This helped me think through the logic step by step. After that, I wrote my solution and used print statements to check that each part worked correctly. I learned how important it is to test edge cases, such as empty lists or lists with only zeros.

During the review and presentation with my partner, I explained my thinking and also listened to their explanation. It was helpful to see a different way of solving the same problem. We both used similar logic, but some small differences in variable names and structure made me realize how code readability matters.

I also reviewed my partner’s code by tracing one of their examples step by step. This helped me understand how their code worked and showed me how to explain code in simple words. Giving

feedback helped me practice communication, and I also learned from the feedback my partner gave me.

Overall, this process helped me become more confident in explaining algorithms and reviewing code in a respectful and useful way.

## Evaluation Criteria

We are looking for the similar points as Assignment 1

- Problem is accurately stated
- New example is correct and easily understandable
- Correctness, time, and space complexity of the coding solution
- Clarity in explaining why the solution works, its time and space complexity
- Quality of critique of your partner's assignment, if necessary

## Submission Information

🚨 **Please review our Assignment Submission Guide** 🚨 for detailed instructions on how to format, branch, and submit your work. Following these guidelines is crucial for your submissions to be evaluated correctly.

### Submission Parameters:

- Submission Due Date: `HH:MM AM/PM - DD/MM/YYYY`
- The branch name for your repo should be: `assignment-2`
- What to submit for this assignment:
  - This Jupyter Notebook (assignment_2.ipynb) should be populated and should be the only change in your pull request.
- What the pull request link should look like for this assignment: `https://github.com/<your_github_username>/algorithms_and_data_structures/pull/`
  - Open a private window in your browser. Copy and paste the link to your pull request into the address bar. Make sure you can see your pull request properly. This helps the technical facilitator and learning support staff review your submission easily.

Checklist:

- [ ] Created a branch with the correct naming convention.
- [ ] Ensured that the repository is public.
- [ ] Reviewed the PR description guidelines and adhered to them.
- [ ] Verify that the link is accessible in a private browser window.

If you encounter any difficulties or have questions, please don't hesitate to reach out to our team via our Slack at `#cohort-6-help`. Our Technical Facilitators and Learning Support staff are here to help you navigate any challenges.

In [ ]: